

这绝对是一个可以冲击满分的硬核《人工智能导论》期末大作业。结合前沿的 AI Agent 架构理念、桌宠的交互表现力，以及你熟悉的系统底层安全机制，这个“数字生命+自动化公司”的综合体不仅具有极高的工程价值，也充满了极客的浪漫。

作为全栈架构师，我将为你梳理出一份面面俱到、由底向上的**全流程开发与落地指南**。我们将采用敏捷开发（Agile）的思路，将整个庞大的系统拆分为五个阶段（Sprints），并明确每一个环节的技术选型、核心代码逻辑与避坑指南。

架构总览：四层物理与逻辑隔离系统

在写下第一行代码前，整个团队必须在脑海中建立起清晰的四层架构图：

- 表现层 (Frontend)**：负责卖萌、渲染 Live2D、捕获用户点击与语音播放。
- 桥接层 (Middleware)**：严格的 WebSocket 强类型 JSON 数据流总线。
- 多智能体中枢 (Agent Brain)**：基于 LangGraph 的状态机，实现“前台老板娘”与“后台牛马车间”的精神隔离。
- 原生安全沙箱 (Native Environment)**：应用层的严格路径约束与人类在环（HITL）审批拦截系统。

第一阶段：视觉基座与桌面通联 (Sprint 1)

目标： 让桌宠在 Windows 桌面上“活”起来，并建立稳定的通信信道。

技术选型： Electron + Vite + React/Vue + Live2D Cubism Web Framework + WebSocket。

详细 workflow：

- Electron 视窗初始化：**
 - 在主进程（Main Process）中，配置窗口为无边框（Frameless）和全透明背景（Transparent）。
 - 技术关键点：** 完美调用 Windows 原生 API，精准计算 Live2D 像素的 Alpha 通道。在前端维护一个机制，只在模型实体（Alpha > 0）区域拦截鼠标点击，空白处必须完美穿透到底层桌面。
- Live2D 渲染与自动化循环：**
 - 在渲染进程中跑通 PixiJS 和 Live2D 模型。

- 实现底层的“肉体本能”：挂载 JavaScript 循环，实现自动眨眼、轻微的呼吸浮动，以及获取全屏幕的鼠标坐标来实现视线跟随。

3. 建立 WebSocket 总线：

- 定义好前后端通信的三大基石数据包格式：
 - Status_Update：后端打工状态更新（如 {"action": "typing", "emotion": "focus"}），前端收到后立刻触发对应的 Live2D 动作和表情。
 - Roleplay_Dialogue：老婆说的话，用于触发气泡UI和后续的语音。
 - Auth_Request：高危操作请求，需前端弹出带有老婆紧张/严肃表情的审批框。

第二阶段：铸造原生安全防线 (Sprint 2)

目标：在不使用 Docker 的前提下，利用 Python 打造一个无法被越权的“虚拟执行沙箱”。

技术选型：Python pathlib, subprocess, re (正则表达式)。

详细工作流：

不要急着接大模型，先把安全基建打好。以你做 CTF PWN 题的直觉，堵住任何可能的 RCE 漏洞。

1. 严格的目录约束 (Path Traversal 防御)：

- 在宿主机创建一个专属的 workspace 文件夹。
- 编写 safe_write_file 和 safe_read_file 工具函数。使用 pathlib.Path.resolve() 获取真实绝对路径。
- **核心防线：**强制检查目标路径是否以 WORKSPACE_DIR 开头。如果尝试跳出该目录（例如写入系统关键目录），直接返回 Permission Denied。

2. 敏感命令黑名单：

- 在底层的 safe_execute_command 函数中，硬编码正则表达式拦截黑名单。
- 直接阻断诸如 del, rmdir /s, format, reg, powershell 等破坏性命令，防止大模型幻觉或用户手滑造成灾难。

3. 资源消耗限制：

- 使用 subprocess.run 执行代码（例如编译 C++ 文件）时，强制加入 timeout 参数（如 10 秒）。超时直接 Kill 进程，防止死循环卡死宿主机系统。

第三阶段：构建“身心分离”的 LangGraph 大脑 (Sprint 3)

目标： 彻底解决大模型“上下文腐化”问题，实现角色扮演与硬核编码的双线并行。

技术选型： Python + LangGraph + 高智商 LLM API (如 DeepSeek-V3 / Claude 3.5 Sonnet)。

详细工作流：

1. 手术级改造图状态 (State Definition):

- 使用 TypedDict 定义全局状态，将“前端展示状态”（如 user_intent, chat_history）和“后台工程状态”（如 tasks_list, current_code, raw_error）完全隔离开。

2. A. 路由门卫 (Intent Router Agent):

- 作为入口点，不保留任何历史记录。只负责将用户自然语言分类为 chat 或 coding，并通过条件边决定流向。

3. B. 表现代理 (Roleplay Agent - 老板娘):

- 拥有独立的 Context。她的输入只有 user_intent 和后台的 UI 状态。
- 设定高傲但关心人的人设（如：“如果连这么简单的指针都会报错...我会盯着后台重写的”）。她的上下文永远不接触 C++ 模板报错等脏数据。

4. C. PM Agent 与 Coder Agent (后台车间):

- **PM Agent:** 接收任务，拆解出 Tasks_List 并休眠。
- **Coder Agent:** 无菌手术室。系统提示词只包含项目背景和当前 Task。不读取任何全局报错历史，只负责吐出纯净的代码并更新 current_code。

第四阶段：防腐化机制与人类在环 (Sprint 4)

目标： 应对编译报错，实现局部的 Bug 修复与高危权限的交互式审批。

技术选型： LangGraph Send API (Map-Reduce), LangGraph interrupt 机制。

详细工作流：

1. Tool Executor 与 QA 过滤器:

- **Executor:** 读取 current_code 进行本地编译（如生成 C++ 测试脚本）。若失败，将长篇 stdout/stderr 写入 raw_error。
- **QA Agent:** 污染过滤器。不写代码，只读取几千字的报错（如 STL 迭代器失效），提炼出人类可读的短摘要（error_summary），并显式地从状态中彻底清除 raw_error，防止污染扩散。

2. 动态孵化 Debugger (究极防腐化秘诀):

- 使用 LangChain 的 Send API。QA 提取摘要后，不流回 Coder，而是动态判定拉起一个专属的 Debugger Agent 实例。
- 只将“错误摘要”和“出错代码”发送给 Debugger。Debugger 修完 Bug 后更新 current_code，随后这个节点及其上下文直接销毁，做到“即用即抛”，确保 AI 大脑永远干净。

3. Human-in-the-Loop 交互确认:

- 在 LangGraph 的执行路径上，针对 Tool Executor 的终端命令执行设置挂起（interrupt）。
- 当遇到如 g++ test.cpp -o test.exe 的命令时，后端暂停，通过 WebSocket 向前端发送 Auth_Request。
- 前端老婆表情切换为询问状态并弹出确认框。用户点击【允许】后，前端回传授权，后端流程继续执行。

第五阶段：多模态注入与体验打磨 (Sprint 5)

目标： 注入灵魂，完成最终的成品级润色。

技术选型： Web Audio API, TTS (如 VITS, Kokoro)。

详细工作流：

1. 动态口型同步 (Lip-Sync):

- 接入高质量的 TTS 语音合成，将老娘板生成的文本转化为音频流。
- 在前端播放时，利用浏览器的 Web Audio API 抓取实时的音频音量包络（Volume Envelope）。将该数值通过数学映射，直接绑定到 Live2D 模型的 ParamMouthOpenY（嘴巴张开度）上，实现自然的“说话”效果。

2. 动画平滑过渡与状态机优化:

- 打磨状态切换时的动画过渡。处理好动作的淡入淡出（Fade-in/out），避免从“思考打字”突然抽搐变成“开心欢呼”。

3. 代码解析正则强化:

- 优化后端解析 Coder Agent 输出流的逻辑。写出强壮的正则表达式，或强制使用 JSON Mode，精准提取纯代码部分，剥离掉 LLM 习惯性输出的“Here is the code...”等

废话前言，确保写入本地的文件能直接编译。

严格按照这五个 Sprints 推进，你们交付的将不再是一个简单的课设脚本，而是一个架构清晰、防线坚固、交互极度优雅的现代化 AI 桌面应用。祝开发顺利，期待这个项目的诞生！